PROJECT SPECIFICATION & ARCHITECTURE GUIDE

SproutGuard: Edge AI Crop Stress Monitor

Complete Technical Execution Lifecycle, Root Cause Analysis & Codebase Reference

Target Hardware: NXP FRDM-MCXW71 (Cortex-M33 @ 96 MHz)

Connectivity: Bluetooth Low Energy 5.3 (GATT Server)

Mobile Platform: Android Native App (Capacitor + React 18)

Git Repository: edge_ai_crop_stress_detection (main)

1. Project Architecture & Mission

SproutGuard is an intelligent edge-computing agritech telemetry platform engineered to compute real-time physiological crop stress directly at the soil level on an ultra-low-power microcontroller. The system eliminates cloud dependence, cellular latency, and ongoing API subscription costs by hosting localized Edge AI inference on the microcontroller and broadcasting real-time biological telemetry via Bluetooth Low Energy (BLE) to a cross-platform mobile application.

Sensing Hardware Rig
FRDM-MCXW71 reading capacitive soil moisture (HW-103 on LPADC0), ambient light intensity (ALS-PT19 on LPADC0 Ch 2B), and ambient temperature & relative humidity (DHT11 single-wire bus on PTC0 / Arduino D2).

Onboard Edge AI Matrix
High-performance C evaluation computing multi-factor agronomic tolerances (Tomato, Basil, Leafy Greens, Succulent) evaluating heat shock, water deficit, and solar radiation within 11.8 ms.

BLE Telemetry Server
Custom Wireless UART GATT service (UUID 01ff0100...) broadcasting clean newline-delimited JSON notification packets every 2 seconds via periodic OSA Timer Manager tasks.

Mobile Application
Modern React 18 dashboard packaged with Capacitor. Features line-buffered packet reassembly, safe-area layout compliance, radial stress gauges, and real-time biological remediation instructions.

2. Step-by-Step Execution Lifecycle

From physical power-on to live UI rendering, the system executes through six distinct phases:

Step 1: MCU Power-On, Peripheral Clock Gating & Pin Muxing

On reset, main() initializes the 96 MHz FRO core clock. Clocks for Port A, Port B, Port C, and GPIO peripherals are unmasked. Cortex-M33 DWT cycle counter (DWT->CYCCNT) is activated to deliver sub-microsecond timing accuracy. LPADC0 is initialized with a 48 MHz FRO clock and 3.3V reference voltage (kLPADC_ReferenceVoltageAlt1). Analog pins (PTD1, PTD2, PTD3, PTA4) are set to kPORT_PinDisabledOrAnalog to disconnect digital input buffers.

Step 2: BLE Host Initialization & Wireless UART Advertisement

The NXP BLE host initializes the controller interface, connection manager, and GATT database. Characteristic 01ff0101-ba5e-f4ee-5ca1-eb1e5e4b1ce0 is registered with Notify and Read properties, and its CCCD descriptor (0x2902) is exposed with open permissions. Pairing and bonding flags are cleared (gAppUsePairing_d = 0). The radio broadcasts connectable undirected advertising packets tagged with the complete local name **'NXP_WU'**.

Step 3: Periodic Sensor Acquisition & Edge AI Inference Tick

An OSA Timer Manager task fires every 2 seconds (CropTelemetryTimerCallback):
• **DHT11 Digital Read:** Pulses PTC0 LOW for 20 ms, releases HIGH for 30 µs, switches to input with internal pull-up, and samples 40 bit-pulses with global interrupts disabled. Verifies checksum byte.
• **Soil Moisture LPADC:** Samples Channel 5 (PTD1/A0) and Channel 6 (PTD2/A1). Converts raw millivolts into calibrated soil percentage (470 mV dry = 0%, 180 mV wet = 100%).
• **Light Intensity Lux:** Samples phototransistor on ADC0 Channel 2 Side B, mapping millivolts to Lux.
• **Edge AI Computation:** CalculateCropStressScore() evaluates multidimensional crop tolerance profiles to generate a real-time stress probability score (0.00 to 1.00).

Step 4: Safe JSON Compilation & ATT MTU Chunk Dispatch

To bypass embedded newlib-nano floating-point limitations, values are formatted via integer whole and fractional tenths digits into: {"soil":%d.%d,"temp":%d.%d,"hum":%d.%d,"lux":%lu,"stress":%d.%d}\r\n.
The string is written to the GATT database attribute. If a central is connected, BleApp_SendUartStream() segments the string into ATT MTU-compatible chunks (typically 20 bytes) and transmits them sequentially via GattServer_SendInstantValueNotification().

Step 5: Mobile App Discovery, Connection & CCCD Subscription

When the user launches SproutGuard, bleManager.js uses @capacitor-community/bluetooth-le to scan specifically for NXP_WU. The client initiates a direct GATT connection, discovers the Wireless UART service, and automatically writes 0x0001 to the CCCD descriptor, instructing the MCU to start pushing notifications.

Step 6: Stream Reassembly, Resilient Parsing & UI Rendering

Incoming packet chunks are accumulated in a FIFO buffer (rxBuffer). When a newline (\n) is detected, the line is sanitized of broken tokens and parsed via JSON.parse(). A secondary regex parser acts as a fail-safe. Valid values update React state hooks in App.jsx, dynamically populating the Soil, Temp, Humidity, and Light cards, the stress gauge, and the agronomic diagnosis panel.

3. Technical Challenges & Engineering Solutions

During development and field debugging, several complex bottlenecks were encountered and resolved:

Symptom & Error	Root Cause Diagnosis	Permanent Resolution Implemented
Android OS Settings Connection Rejection: *'Failed to connect to NXP_WU. The connection request was rejected.'* (HCI 0x113)	Pairing via Android System Bluetooth Settings triggers Classic Audio (A2DP) / HID profile lookups. The board is an unauthenticated BLE GATT server; pairing negotiation failure provoked an immediate L2CAP disconnect.	Disabled pairing/bonding completely in MCU firmware (gAppUsePairing_d = 0, gAppUseBonding_d = 0). Enforced app-exclusive connection routing via Capacitor BLE client with visual user warnings.
Blank Widgets Lockout: App shows *'CONNECTED - LIVE'*, but all 5 dashboard cards remain pinned at --.	**Newlib-Nano snprintf Bug:** In standard GCC ARM Embedded builds, snprintf() omits floating-point %f formatting by default to save flash. The MCU was transmitting {"soil":,"temp":,"hum":,"lux":859,"stress":}, crashing JSON.parse() with a SyntaxError every cycle.	Refactored MCU telemetry formatter to compute integer whole and fractional decimal digits explicitly (%d.%d). In the mobile frontend, added pre-parse JSON colon sanitization and robust regex fallback matching.
GATT Collisions & Link Drops: BLE connection drops after 6–10 seconds; log shows [BLE Native] read error.	A client-side 3-second polling interval executed concurrent GattRead operations on the exact characteristic that was simultaneously streaming periodic notifications, provoking GATT queue collisions in the Android stack.	Decommissioned the automatic 3-second background polling timer in bleManager.js. The link now relies exclusively on clean 2-second MCU push notifications, keeping GATT read available solely for manual sync.
Duplicate Incomplete Packets: Incoming notification stream showed corrupted, interleaved JSON lines arriving out of sequence.	In wireless_uart.c, after chunking the string through GattServer_SendInstantValueNotification(), an extraneous call to GattServer_SendNotification() was executed, inserting a redundant truncated packet into the queue.	Removed the duplicate notification call, ensuring strictly ordered, single-transmission packet delivery across MTU boundaries.
DHT11 Timing & Hardware Bus Glitches: Zero temperature and humidity readings or intermittent checksum failures.	Microsecond timing errors caused by RTOS interrupts preempting the single-wire timing loop, coupled with floating input pins lacking unmasked digital buffers in the port PIDR register.	Wrapped the 40-bit transmission window in __disable_irq() and __enable_irq(). Explicitly unmasked input buffers via GPIO_PortInputEnable(), enabled internal pull-up resistors, and utilized hardware DWT cycle counters.

4. Main Source Files & Functional Reference

File: crop-stress-frdm-mcxw71/source/wireless_uart.c

The central firmware orchestration engine. Implements the 2-second telemetry timer (CropTelemetryTimerCallback), coordinates hardware sensor polling across LPADC and DHT11, encodes integer-decimal JSON strings, writes to GATT database attributes, and handles BLE chunked notification dispatch via BleApp_SendUartStream().

File: crop-stress-frdm-mcxw71/source/board_adc.c

Hardware abstraction driver for analog crop sensing. Configures LPADC0 with 48 MHz FRO clock, auto-calibration, and 3.3V reference voltage. Implements BOARD_ReadSoilAdcMv(), CalculateSoilMoisturePercent(), and BOARD_ReadAmbientLightLux().

File: crop-stress-frdm-mcxw71/source/dht11.c

High-precision bit-banged digital single-wire driver for the DHT11 sensor on PTC0 (Arduino D2). Uses Cortex-M33 DWT cycle counter for sub-microsecond pulse measurement, wraps the 40-bit read in interrupt disabling, verifies 8-bit checksums, and caches last-valid data.

File: crop-stress-frdm-mcxw71/source/crop_stress.c

Onboard Edge AI Inference implementation. Evaluates multi-variable agronomic stress functions against species tolerance profiles (Tomato, Basil, Leafy Greens, Succulent), producing a real-time composite stress probability score within 11.8 ms.

File: crop-stress-mobile/src/services/bleManager.js

Unified cross-platform BLE communication service. Bridges Capacitor Native BLE (Android) and Web Bluetooth API. Implements chunked packet buffering, line-based JSON splitting, syntax sanitization, regex fallback parsing, and state event emission.

File: crop-stress-mobile/src/App.jsx

Top-level React interface component. Manages connection state machine, registers BLE telemetry listeners, applies resilient partial state updates, calculates dynamic crop health diagnosis, and renders the header, stress gauge, and sensor grid components.

Final Hardware & Mobile Verification:

The firmware binary was compiled with Ninja and flashed onto the target FRDM-MCXW71 board via LinkServer. The Android application was built via Gradle and deployed to the connected physical device (OPPO Reno15c 5G). Live data verification confirmed continuous real-time streaming: **Soil Moisture: 98.6%**, **Air Temperature: 34.0°C**, **Humidity: 76.0%**, **Sunlight: 30 lx**, and **Stress Score: 0.21**.